



7



8



2000 sp

نظري

RB Informatics; 10-7-2026

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الثالثة

CYK

د. باسم قصيبة



اللغات الصورية

بسم الله الرحمن الرحيم

طرق الاشتقاق:

- ❖ Top-Down (من الأعلى إلى الأسفل): يبدأ الاشتقاق من رمز البداية (Start Symbol: S) ويستمر تبديل القواعد للوصول إلى الكلمة المقبولة
- ❖ Bottom-Up (من الأسفل إلى الأعلى): تبدأ العملية من النهايات (Terminals) وتحاول تجميعها وتطبيق القواعد بشكل عكسي للوصول في النهاية إلى رمز البداية S إذا تم الوصول إليه، فالكلمة مقبولة.

خوارزمية CYK (Cocke-Younger-Kasami Algorithm):

تسمح بتحقيق تحليل نحوي (Syntax Analysis)، وهذا يعني تحديد إذا ما كانت الكلمة تنتمي إلى لغة مولدة بواسطة CFG و هي تعتمد على طريقة Bottom-Up. تعتمد هذه الخوارزمية على أسلوب البرمجة الديناميكية (Dynamic Programming - DP) حيث تشبه عملها مبدأ خوارزمية حساب متتالية فيبوناتشي

مثال:

لحساب $fib(10)$ هي $fib(8) + fib(9)$

في البرمجة الديناميكية DP تبدأ بها من الواحد $fib(1)$ وتخزينه في كل مرة للوصول إلى الرقم المطلوب $fib(10)$ ، بالتالي يصبح التعقيد خطياً $O(n)$ ولكن لكي تعمل خوارزمية CYK يجب أن تكون القواعد بصيغة تشومسكي الطبيعية (Chomsky Normal Form - CNF).

الشروط الواجب توافرها في القواعد لتكون بصيغة CNF:

1. أن ينتج المتغير غير الطرفي متغيرين غير طرفيين فقط (مثال: $A \rightarrow BC$)
2. أن ينتج عن المتغير غير الطرفي نهاية (Terminal) فقط (مثال: $A \rightarrow a$)
3. أن تكون القواعد خالية تماماً من توليد السلاسل الفارغة (مثل: $A \rightarrow \epsilon$)
4. أن تكون خالية من القواعد الأحادية (Unit Rules) (مثل: $A \rightarrow B$)

خطوات تحويل القواعد إلى صيغة تشومسكي (CNF):

1. حذف القواعد الفارغة: $A \rightarrow \epsilon$
2. حذف القواعد الأحادية: $A \rightarrow B$
3. النهايات الطرفية المدمجة: في حال وجود رموز طرفية مدمجة، يتم استبدالها بمتغير جديد.

مثال: $A \rightarrow X_0Z$ $\leftarrow$ $X_0 \rightarrow a$ $\leftarrow$ $A \rightarrow aZ$

4. القواعد الطويلة (أكثر من متغيرين): تجزئة القواعد الطويلة واستبدال كل متغيرين بمتغير واحد.

مثال: $A \rightarrow CZ0$ $\leftarrow$ $Z0 \rightarrow BW$ $\leftarrow$ $A \rightarrow CBW$

القواعد الأصلية:

$A \rightarrow AT | bT | b|bU | abW$
 $T \rightarrow bT | b$
 $C \rightarrow bT | b | UC$
 $W \rightarrow TUC | UC | TC | bT | b$
 $U \rightarrow bU | b | abW$

الحل:

أولاً: التخلص من الرموز الطرفية (Terminals) في القواعد التي تولد أكثر من رمز.

$X_a \rightarrow a$
 $X_b \rightarrow b$

تصبح قواعد:

$A \rightarrow AT | X_bT | b | X_bU | X_a X_bW$
 $T \rightarrow X_bT | b$
 $C \rightarrow X_bT | b | UC$
 $W \rightarrow TUC | UC | TC | X_bT | b$
 $U \rightarrow X_bU | b | X_a X_bW$

ثانياً: تقصير القواعد التي تحتوي على أكثر من رمزين.

$Z1 \rightarrow X_bW$
 $Z2 \rightarrow UC$

تصبح القواعد بالشكل النهائي:

$A \rightarrow AT | X_bT | b | X_bU | X_a Z1$
 $T \rightarrow X_b T | b$
 $C \rightarrow X_bT | b | UC$
 $W \rightarrow T Z2 | UC | TC | X_bT | b$
 $U \rightarrow X_bU | b | X_a Z1$

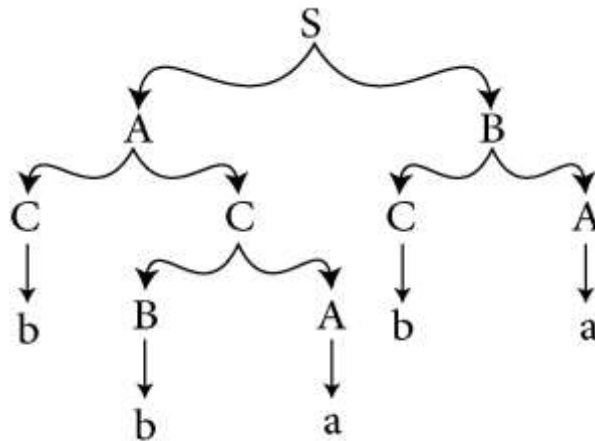
مثال شامل على تطبيق خوارزمية CYK :

لتكن القواعد معطاة جاهزة بصيغة CNF كتالي

$$\begin{aligned} S &\rightarrow AB \mid BB \\ A &\rightarrow CC \mid AB \mid a \\ B &\rightarrow BB \mid CA \mid b \\ C &\rightarrow BA \mid AA \mid b \end{aligned}$$

ولتكن الكلمة المطلوب اختبارها: (طول الكلمة = 5) bbaba

بالبدية لنشتق الكلمة بالطريقة Top-Down للتأكد من أن الكلمة مقبولة



الخطوة الأولى :

كتابة مجموع القواعد غير الطرفية (Non-Terminals) التي تولد أحرف الكلمة، تضعهم ضمن خلايا لكل حرف:

$$\text{Cell}(1, 1) = \{B, C\}$$

حيث: {جميع القواعد التي تولد الاحرف} = (مكان بداية هذه الأحرف , عدد الأحرف التي أحاول توليدها) Cell
في المثال السابق كان الحرف هو b

$$\text{Cell}(1, 2) = \{B, C\}$$

$$\text{Cell}(1, 3) = \{A\}$$

$$\text{Cell}(1, 4) = \{B, C\}$$

$$\text{Cell}(1, 5) = \{A\}$$

الخطوة الثانية :

كتابة مجموعة القواعد الغير طرفية التي تولد حرفين : خلال CC من خلال BB

$$\text{Cell}(2, 1) = \{S, B, A\} \rightarrow \text{(اول و ثاني حرف)}$$

$$\text{Cell}(2, 2) = \{B, C\} \rightarrow \text{(ثاني و ثالث حرف)} \{CA, BA\}$$

لايوجد قاعدة في S أو A نستطيع توليد "ba"

$$Cell(2,3) = \{S, A\}$$

$$Cell(2,4) = \{B, C\}$$

الخطوة الثالثة :

كتابة مجموع القواعد الغير طرفية التي تولد 3 أحرف :

Cell (3 , 1) يمكننا توليد الخلية بطريقتين :

اما بتوليد أول حرفين ل cell (2 , 1) ثم ثالث حرف ل cell (1 , 3)
أو بتوليد أول حرف cell (1,1) ثم الحرفين الثاني والثالث cell (2 , 2)

بعد تطبيق كل طريقة نقوم باجتماع لنتائج التوليد , ثم جمع نتائج توليد الطريقتين

$$Cell(3,1) = cell(1,1) U cell(2,2) + cell(2,1) U cell(1,3)$$

(1) نقوم بأخذ جميع الاحتمالات من خلال الضرب الديكارتي

$$Cell(1,1) U cell(2,2) = \{C, B\} U \{B, C\} \rightarrow CB, CC, BB, BC$$

(2) نبحث في القواعد التي تولد هذه التشكيلات :

CB : لاتوجد أي قاعدة تولد هذا المقطع

تولدها القاعدة CC:A

BB : تولدها القاعدة S,B

لاتوجد أي قاعدة تولد هذا المقطع BC :

(3) نقوم بجمع للقواعد

$$\rightarrow Cell(1,1) U cell(2,2) = \{S, B, A\}$$

$$Cell(2,1) U cell(1,3) = \{S, B, A\} U \{A\} \rightarrow SA, BA, AA$$

لا يوجد قاعدة تولد هذا المقطع SA :

تولدهم ال C فقط : BA , AA

ملاحظة: لم نأخذ القاعدة A لانها تولد القاعدة AB وليس BA (الترتيب مهم).

$$\rightarrow Cell(2,1) U Cell(1,3) = \{C\}$$

$$cell(1,1) U cell(2,2) + cell(2,1) U cell(1,3)$$

$$= \{S, B, A\} + \{C\} = \{S, B, C, A\}$$

نتابع بنفس الخطوات لبقية الخلايا

$$Cell(3,2) = Cell(1,2) U Cell(2,3) + Cell(2,2) U Cell(1,4)$$

$$Cell(1,2) U cell(2,3) = \{C, B\} U \{A, S\} = CA CS BA BS = \{B, C\}$$

$$Cell(2,2) U cell(1,4) = \{B, C\} U \{C, B\} = BC BB CC CB = \{B, S, A\}$$

$$\rightarrow Cell(3,2) = \{B, S, A, C\}$$

$$Cell(3,3) = Cell(1,3) U Cell(2,4) + Cell(2,3) U Cell(1,5)$$

$$Cell(1,3) U Cell(2,4) = \{A\} U \{B, C\} = AB AC = \{A\}$$

$$Cell(2,3) U Cell(1,5) = \{A, B\} U \{B, C\} = AB AC BB BC = \{A, S, B\}$$

$$\rightarrow Cell(3,3) = \{A, S, B\}$$

الخطوة الرابعة :

كتابة مجموع القواعد في الطرفية التي تولد 4 أحرف:

Cell (4,1) يمكننا توليد هم بثلاث طرق
 اما توليد حرف ثم 3 أحرف
 أو توليد حرفين ثم حرفين
 أو توليد 3 أحرف ثم حرف

$$\begin{aligned}
 &Cell(4,1) = \\
 &Cell(1,1) \cup Cell(3,2) = \{C, B, \} \cup \{B, C, S, A\} = \{A, B, S, C\} \\
 &+ Cell(2,1) \cup Cell(2,3) = \{A, S, B\} \cup \{A, S\} = \{C\} \\
 &+ Cell(3,1) \cup Cell(1,4) = \{S, B, C\} \cup \{C, B\} = \{B, S, A\} \\
 &\rightarrow Cell(4,1) = \{A, B, S, C\} \\
 &Cell(4,2) = \\
 &Cell(1,2) \cup Cell(3,3) = \{C, B, \} \cup \{A, S, B\} = \{B, S, C\} \\
 &+ Cell(2,2) \cup Cell(2,4) = \{B, C\} \cup \{C, B\} = \{S, B, A\} \\
 &+ Cell(3,2) \cup Cell(1,5) = \{B, S, C, A\} \cup \{C, B\} = \{B, S, A\} \\
 &\rightarrow Cell(4,2) = \{B, C, S, A\}
 \end{aligned}$$

الخطوة الخامسة:

كتابة مجموع القواعد في الطرفية التي تولد 5 أحرف :

يمكننا توليدهم ب 40 طرق
 توليد حرف ثم 4 أحرف
 توليد حرفين ثم 3 أحرف
 توليد 3 أحرف ثم حرفين
 توليد 4 أحرف ثم حرف

$$\begin{aligned}
 &Cell(5,1) = \\
 &Cell(1,1) \cup Cell(4,2) = \{C, B, \} \cup \{B, C, S, A\} = \{A, B, C, S\} \\
 &+ \\
 &Cell(2,1) \cup Cell(3,3) = \{A, B, S\} \cup \{A, B, S\} = \{C, A, B, S\} \\
 &+ \\
 &Cell(3,1) \cup Cell(2,4) = \{S, B, C\} \cup \{B, C\} = \{S, B, A\} \\
 &+ \\
 &Cell(4,1) \cup Cell(1,5) = \{A, S, C, B\} \cup \{C, B\} = \{A, B, S\} \\
 &\Rightarrow Cell(5,1) = \{A, B, C, S\}
 \end{aligned}$$

ملاحظة: وجود الـ S في الخلية الأخيرة دلالة على أن الجملة يمكن توليدها من S أي الكلمة مقبولة (معربة)
 وإذا لم تظهر فهذا يعني أن الكلمة غير مقبولة (غير معربة).

لنتبع كيفية الاشتقاق على طريقة Bottom - Up:

*القاعدة للتتبع: نقوم بتفكيك الكلمات بأخذ آخر حرف ونربطه بالقاعدة التي تولده إلى أن نصل إلى S.

نفك الكلمة $bbaba$ إلى قسمين:

$$X: bbab, A: a$$

ثم نفك X نجد:

$$Y: bba, B: b$$

نفك Y :

$$Z: bb, A: a$$

ونجد أيضاً أن bb يمكن توليدها من المقطع BB أي من القاعدة S ، وهذا يعني أن $Z = S$.

ملخص خوارزمية CYK:

نكتب:

* ماذا لو تتبعنا مسار الاشتقاق ووجدنا أن S تأتي من طريقين مختلفين؟

هنا نسمي اللغة بلغة غامضة وهذا ترفضه لغات البرمجة، ف لغة البرمجة يجب أن لا تكون غامضة.

تعريف اللغات خارج السياق (CFG) Context Free Grammar:

تعرف بأربع مكونات:

1. Variables $V \rightarrow$ هي الحروف الكبيرة التي تمثل الرموز وتولد كلمات فرعية، و تأتي من جهة اليسار.
2. Terminals $T \rightarrow$ هي الحروف التي تشكل كلمات اللغة، لا يمكن اشتقاق شيء منها، وتكتب عادة بالحروف الصغيرة و لا يمكن ان تأتي من جهة اليسار ابدا .
3. Rules $R \rightarrow$ القواعد التي تصف كيفية استبدال المتغيرات بالمتغيرات الأخرى أو بالنهايات الطرفية.
4. Start Symbol $S \rightarrow$ وهو الرمز الذي تبدأ منه عملية الاشتقاق.

القواعد الغامضة (Ambiguous Grammar):

مثال على قواعد الغموض:

ليكن لدينا القاعدة:

$$E \rightarrow E + E \mid E * E \mid ID$$

وليكن التعبير:

$$ID + ID * ID$$

هنا القاعدة غامضة لأننا يمكن أن نشق التعبير بطريقتين:

- إما الجمع أولاً ثم الضرب: $(ID + ID) * ID$
- أو الضرب أولاً ثم الجمع: $ID + (ID * ID)$

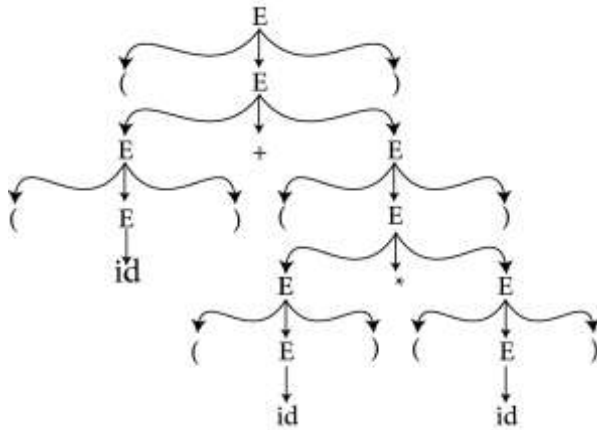
وليكن $ID = 3$ بالطريقة الأولى تطلع النتيجة 18

أما بالطريقة الثانية تطلع النتيجة: 12

بالتالي أصبح لدينا تضارب في النتائج (أي اللغة غامضة)، ولذلك ترفضها المترجمات (Compilers)؛ لأن المترجم يحتاج إلى شجرة اشتقاق واحدة فريدة.

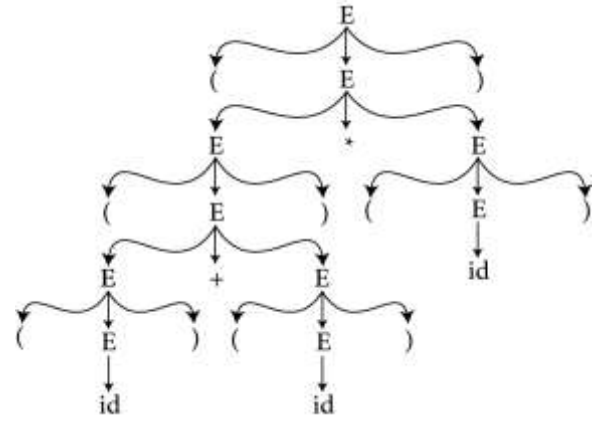
للتأكيد على غموض هذه القواعد سنزيد قاعدة ليس لها تأثير على تركيب التعبيرات

$$E \rightarrow (E) | E + E | E * E | ID$$



((id)+((id)*(id)))

(id+(id*id))



((id+id)*(id))

(id+id)*id

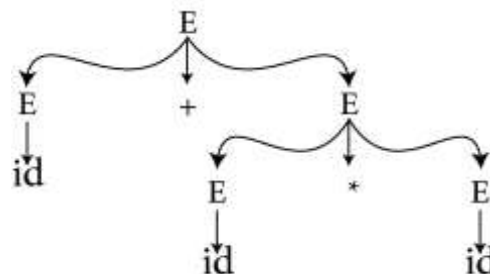
الاشتقاق اليميني واليساري: (Leftmost & Rightmost Derivations)

1. الاشتقاق اليساري: (Leftmost Derivation)

عملية الاشتقاق تكون Left-most إذا كانت تتم باستبدال الـ Non-Terminal من اليسار أولاً.

مثال :

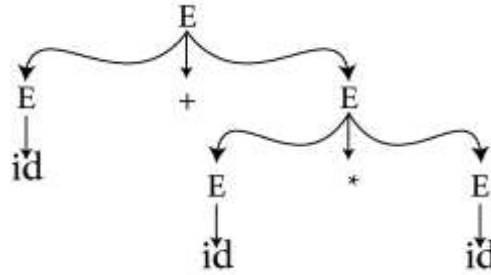
$$E \rightarrow E + E \rightarrow ID + E \rightarrow ID + E * E \rightarrow ID + ID * E \rightarrow ID + ID * ID$$



2. الاشتقاق اليميني: (Rightmost Derivation)

عملية الاشتقاق تكون Right-most إذا كانت تتم باستبدال الـ Non-Terminal من اليمين أولاً.
مثال :

$$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * ID \rightarrow E + ID * ID \rightarrow ID + ID * ID$$



كما نرى فان الاشتقاق اليساري والاشتقاق اليميني له نفس النتيجة وينتج في النهاية نفس الشجرة من أجل نفس الكلمة W والتي هي هنا ID+ID * ID

نلاحظ انه من أجل كل اشتقاق يساري يوجد اشتقاق يميني للكلمة W ويعطيان نفس الشجرة



The End